

## Project - Calculator

### Learning Objectives

- Design and implement a functional calculator using functions and control structures
- Apply modular programming principles by separating concerns into functions
- Handle user input, validate data, and implement error handling

### Engage (5-7 minutes)

A calculator is one of the first real programs many developers build. It combines everything learned so far: functions for each operation, loops for continuous calculation, conditionals for choosing operations, and string handling for input processing. Today we bring all concepts together to build a complete, working calculator from scratch.

### Explore (10-12 minutes)

Before writing code, plan the calculator on paper. What operations should it support? How will the user select an operation? What happens when the user enters invalid input like “abc” instead of a number? Sketch the program structure with separate functions for add, subtract, multiply, divide, and a main loop that handles user interaction. List at least five edge cases to test.

### Explain (10-12 minutes)

A well-designed calculator uses modular programming—each operation is a separate function, making the code organized and easy to extend. The main program loop handles user interaction, input validation, and calling the appropriate function. Key design considerations include error handling for division by zero, non-numeric input, and unsupported operations. The code should follow the single responsibility principle, where each function does one thing well.

### Elaborate (8-10 minutes)

Build the calculator in stages: first, implement basic arithmetic functions. Then add a menu system that lets users choose operations. Next, add input validation to reject non-numeric values. Finally, implement a history feature that stores the last 10 calculations. Test thoroughly with edge cases: division by zero, very large numbers, decimal inputs, and invalid menu choices. Refactor your code to improve readability and add comments explaining complex logic.

### Evaluate (5-8 minutes)

Present your calculator to a peer and have them test it with unexpected inputs. Write a brief report covering: the program structure, how errors are handled, three test cases with expected and actual results, and one improvement you would make if given more time. Submit your code with clear function documentation.